DEVSECOPS LAB BOOK · CHAPTER 09

Ansible Configuration Management & Infrastructure Automation

From provisioned virtual machines to fully configured, idempotent infrastructure — using YAML, SSH, and zero agents.

CHAPTER 09

Learning Objectives

- Understand Configuration Management
- Understand IaC vs Configuration as Code
- Install Ansible & configure passwordless SSH
- Build a production-ready Ansible project
- Create inventories & use host groups
- Use variables and facts
- Write Playbooks & create reusable Roles
- Use Templates, Handlers, and Tags
- Use Vault and Galaxy Collections
- Debug Playbooks
- Deploy services automatically
- Prepare infrastructure for later chapters

9.1

What is Configuration Management?

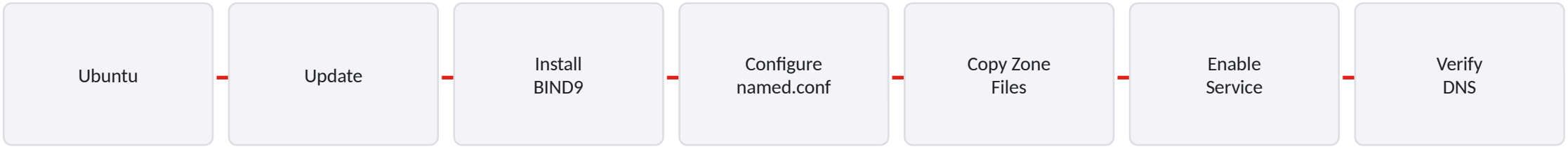
Provisioning asks:
"What infrastructure should exist?"

Configuration Management asks:
"What should be installed and configured inside it?"

Vagrant creates

`dns-01`

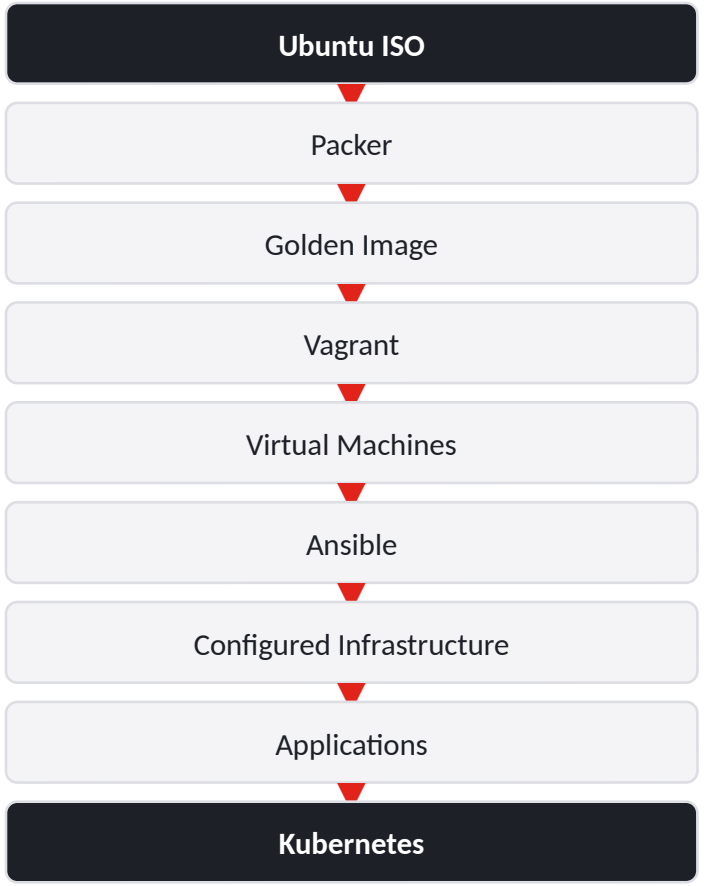
Ansible then configures it step by step:



Everything is described as code.

9.2

The Modern DevOps Workflow



Tool	Responsibility
Packer	Image Creation
Vagrant	Infrastructure Provisioning
Ansible	Configuration Management
Kubernetes	Container Orchestration

Keeping responsibilities separate makes infrastructure easier to maintain.

Why Not Configure Servers Manually?

Imagine twelve servers, configured by hand:

```
ssh ubuntu@10.0.2.10
sudo apt install bind9

ssh ubuntu@10.0.2.11
sudo apt install lldap

ssh ubuntu@10.0.2.14
sudo apt install gitlab

… repeat dozens of times …
```

Manual administration does not scale — every server slowly becomes slightly different.

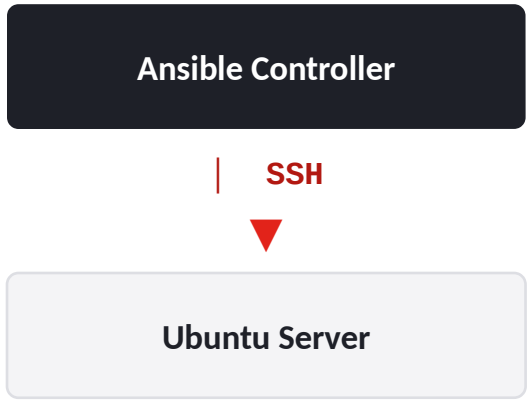
The Ansible Solution

```
ansible-playbook site.yml
```

- Connects to every server
- Installs required software
- Configures services
- Starts daemons
- Verifies state & reports failures

Agentless by Design — Architecture

Ansible requires no agents, no client software, and no additional daemons — it communicates over standard SSH.



Full architecture — one controller, many managed nodes:



Ansible Controller sends instructions — no permanent connection is maintained.

9.6

Why We Chose Ansible

Tool	Language
Ansible	YAML
Puppet	DSL
Chef	Ruby
Salt	Python
CFEngine	DSL

- Easy to learn
- Agentless
- Idempotent
- Cross-platform
- Cloud-native
- Enterprise-ready
- Widely adopted

Today Ansible is one of the most commonly used automation platforms in the DevOps industry.

9.7 – 9.10

Installing Ansible & Preparing SSH

1 · Install & Verify

```
sudo apt update
sudo apt install ansible -y
ansible --version

# ansible [core 2.17.x]
# python version = 3.x
```

2 · Verify SSH Connectivity

```
ssh ubuntu@10.0.2.10
exit
# repeat for every server
```

3 · Generate an SSH Key Pair

```
ssh-keygen -t ed25519
# ~/.ssh/id_ed25519
# ~/.ssh/id_ed25519.pub
```

Private key stays on the controller · public key goes to every node.

4 · Copy the Key to Every Node

```
ssh-copy-id ubuntu@10.0.2.10
ssh-copy-id ubuntu@10.0.2.11
ssh-copy-id ubuntu@10.0.2.12
...

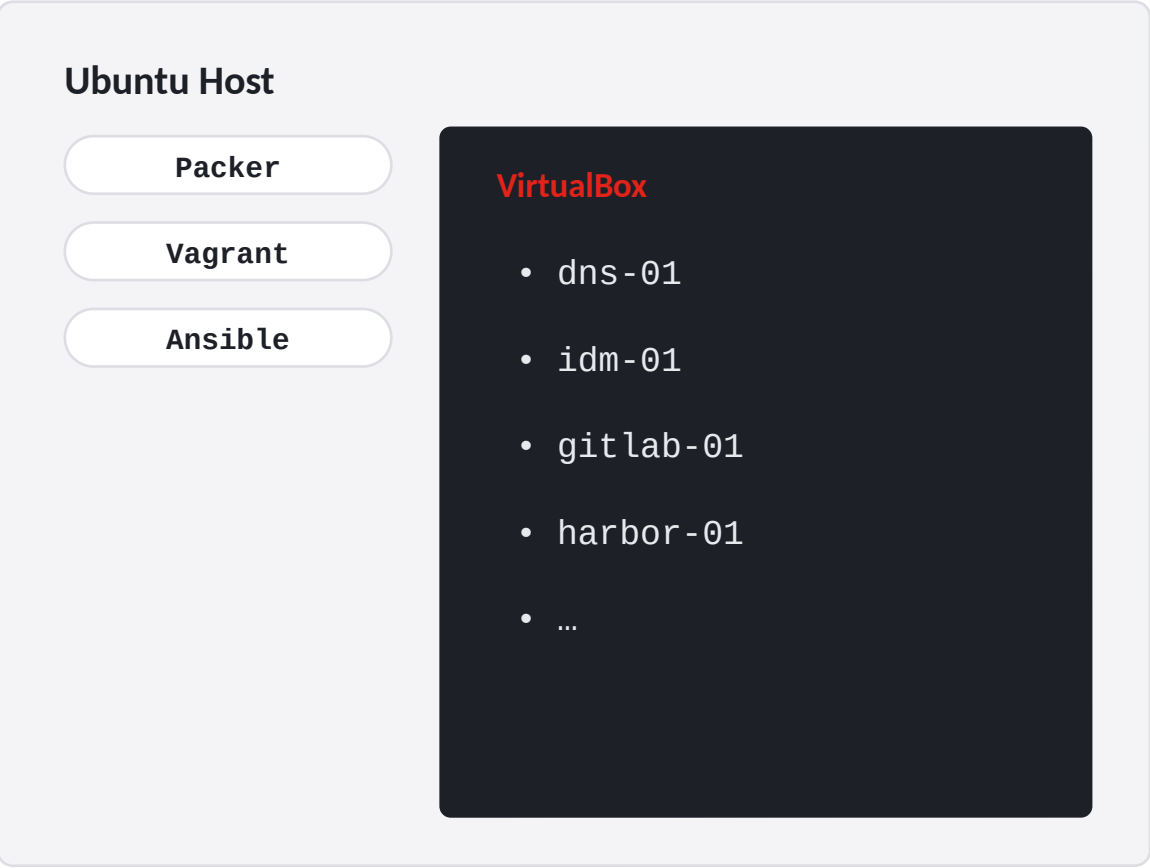
# afterwards: passwordless login
```

9.11

Where Should Ansible Run?

The Ansible Controller runs on the host machine (your Ubuntu workstation) — not inside a VM.

- A single control point for the entire lab
- Easy access to the Git repository
- No need to dedicate a VM solely to Ansible
- Simpler integration with Vagrant & future CI/CD



PART II

Building a Production Ansible Project

From a single monolithic playbook to a scalable, maintainable project layout.

Project Structure

Beginners place everything in one file — professionals separate concerns.

```
ansible/
├── ansible.cfg
├── inventory/production/
│   ├── hosts.ini
│   ├── group_vars/
│   └── host_vars/
├── playbooks/
│   ├── site.yml
│   ├── common.yml
│   └── kubernetes.yml
├── roles/
├── collections/
├── requirements.yml
└── README.md
```

Create the directories:

```
mkdir -p ansible && cd ansible
mkdir -p inventory/production
mkdir -p inventory/production/group_vars
mkdir -p inventory/production/host_vars
mkdir playbooks roles collections
```

This layout follows Ansible best practices and scales from a few servers to hundreds.

Configuring ansible.cfg

```
[defaults]
inventory = inventory/production/hosts.ini
roles_path = roles
collections_paths = collections
host_key_checking = False
stdout_callback = yaml
forks = 10
timeout = 30
remote_user = ubuntu
```

Option	Purpose
inventory	Default inventory
roles_path	Role search path
host_key_checking	Ignore unknown SSH keys
stdout_callback	Cleaner output
forks	Parallel connections
remote_user	Default SSH user

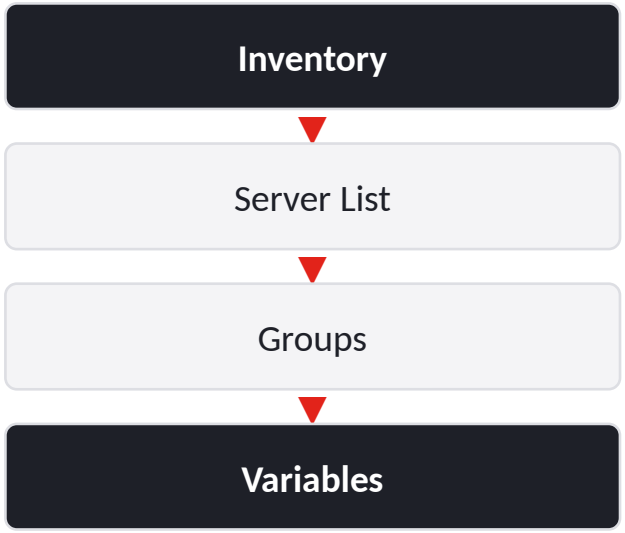
`Verify: ansible-config dump --only-changed`

These settings eliminate the need to repeat command-line arguments.

9.17 – 9.18

Understanding Inventories

An Inventory is the address book of your infrastructure — every playbook begins by reading it.



Static Inventory

- Simple
- Easy to understand
- Perfect for local labs

Dynamic Inventory

- Queries AWS, Azure
- VMware, OpenStack
- GCP automatically

This lab uses a Static Inventory (hosts.ini).

9.19 – 9.21

Creating & Verifying the Inventory

```
[dns]
dns-01 ansible_host=10.0.2.10

[gitlab]
gitlab-01 ansible_host=10.0.2.14

[kubernetes_workers]
k8s-worker-01 ansible_host=10.0.2.21
k8s-worker-02 ansible_host=10.0.2.22

[all:vars]
ansible_user=ubuntu
```

Verify:

```
ansible-inventory --list
ansible-inventory --graph
```

```
@all
|--@dns
|  |--dns-01
|--@gitlab
|  |--gitlab-01
|--@kubernetes_workers
|  |--k8s-worker-01
|  |--k8s-worker-02
```

inventory/production/hosts.ini · servers are grouped by responsibility, not OS.

9.22 – 9.24

Ad-Hoc Commands & Modules

Test connectivity first — the ping module uses SSH, not ICMP:

```
ansible all -m ping

dns-01    | SUCCESS
gitlab-01 | SUCCESS
monitor-01| SUCCESS
```

Run ad-hoc commands:

```
ansible all -a "hostname"
ansible all -a "uname -r"
ansible all -a "df -h"
```

Prefer Modules over shell commands:

```
ansible all -m command -a "uptime"
ansible all -m package_facts
ansible all -m service_facts
```

- Idempotent
- Portable
- Easier to maintain
- More reliable than shell scripts

Gathering & Filtering Facts

Facts: information Ansible automatically discovers about every managed host.

```
ansible all -m setup

{
  "ansible_hostname": "dns-01",
  "ansible_distribution": "Ubuntu",
  "ansible_kernel": "6.8.0",
  "ansible_memtotal_mb": 2048
}
```

Filter facts instead of dumping everything:

```
ansible all -m setup -a "filter=ansible_default_ipv4"
```

Fact	Description
ansible_hostname	Hostname
ansible_distribution	Linux Distribution
ansible_kernel	Kernel Version
ansible_default_ipv4.address	Primary IP Address
ansible_memtotal_mb	Installed Memory
ansible_virtualization_type	Virtualization Platform

9.29 – 9.33

Variables & Precedence

Instead of hardcoding:

```
name: Install Bind9
```

write:

```
name: Install {{ dns_package }}
# dns_package: bind9
```

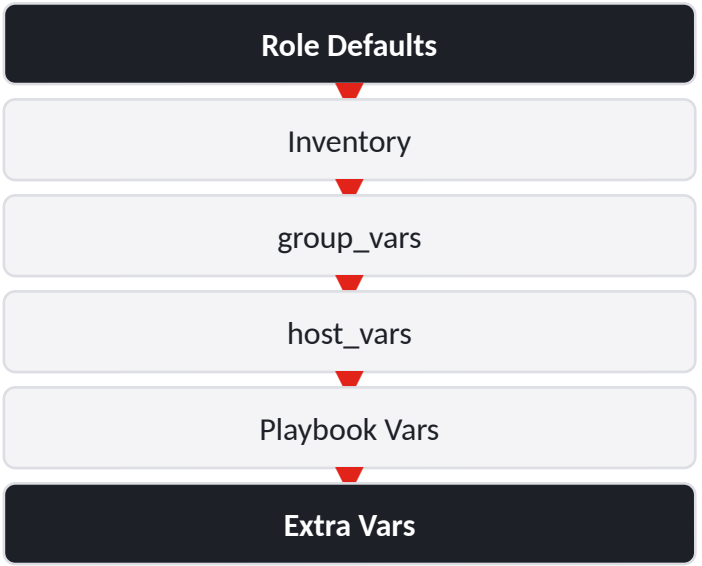
Good names: gitlab_version, vault_cluster_name, dns_forwarders

Avoid: x, abc, test, tmp, var1

group_vars/kubernetes_workers.yml

```
container_runtime: containerd
pod_network_cidr: 10.244.0.0/16
```

Variable Precedence (low → high)



PART III

Playbooks: Tasks, Idempotency & Safety

The entry point for every automation run — and the safeguards that make it trustworthy.

Anatomy of a Playbook

```
---
- name: Install Packages
  hosts: all
  become: true
  tasks:
    - name: Install curl
      apt:
        name: curl
        state: present
```

playbooks/site.yml is the entry point for the entire infrastructure.

Directive	Purpose
name	Description
hosts	Target hosts
become	Privilege escalation
tasks	List of tasks
handlers	Restart services
roles	Execute roles
vars	Variables

A Task performs one well-defined action: install a package, copy a file, start a service.

9.37 – 9.39

Idempotency, Check Mode & Diff Mode

An idempotent task produces the same result no matter how many times it runs.



Check Mode — Dry Run

```
ansible-playbook playbooks/site.yml --check
```

Simulates execution — no changes are made.

Diff Mode

```
ansible-playbook playbooks/site.yml --diff

- PermitRootLogin yes
+ PermitRootLogin no
```

Your First Real Playbook — common.yml

```
---
- name: Configure Common Linux Settings
  hosts: all
  become: true
  tasks:
    - name: Update package cache
      apt: { update_cache: true }
    - name: Upgrade installed packages
      apt: { upgrade: dist }
    - name: Install common packages
      apt:
        name: [curl, wget, vim, git, unzip, tree, htop, jq]
        state: present
```

```
ansible-playbook playbooks/common.yml
```

Every server in the laboratory now shares the same baseline package set.

Ready for the next step:

Organizing this logic into reusable Roles.

PART IV

Ansible Roles: Modular, Reusable Automation

Splitting a 10,000-line playbook into self-contained, service-focused packages.

What is a Role?

A Role is a self-contained automation package — one service, one responsibility.

```
roles/
├── common/
├── dns/
├── idm/
├── vault/
├── gitlab/
├── harbor/
└── kubernetes/
```

Role	Responsibility
common	Base Linux configuration
dns	BIND9 DNS Server
idm	LLDAP + Authentik
vault	HashiCorp Vault
gitlab	GitLab CE
kubernetes	Kubernetes Cluster
monitoring	Prometheus + Grafana

- Modular
- Reusable
- Easier to test
- Easier to maintain

Creating & Understanding Role Structure

Generate the standard layout automatically:

```
ansible-galaxy init roles/common
```

```
roles/common/
├── defaults/
├── files/
├── handlers/
├── meta/
├── tasks/
├── templates/
├── tests/
└── vars/
```

Directory	Purpose
tasks/	Automation tasks
handlers/	Executed only when notified
templates/	Jinja2 templates (.j2)
files/	Static files
defaults/	Lowest-priority variables
vars/	Role-specific variables
meta/	Author, license, dependencies
tests/	Role testing

The Main Task File & Executing the Role

```
# roles/common/tasks/main.yml
---
- name: Update package cache
  ansible.builtin.apt:
    update_cache: true
- name: Install common packages
  ansible.builtin.apt:
    name: [curl, wget, vim, git, jq]
    state: present
```

Every module uses its Fully Qualified Collection Name (ansible.builtin.apt) — a best practice covered later in this chapter.

Wire it into site.yml:

```
---
- name: Configure DevSecOps Lab
  hosts: all
  become: true
  roles:
    - common
```

```
ansible-playbook playbooks/site.yml
```

Every managed node now executes the common role.

Handlers: Restart Only When Needed

Restarting a service on every run wastes time — Handlers fire only when notified.

handlers/main.yml

```
---
- name: Restart SSH
  ansible.builtin.service:
    name: ssh
    state: restarted
```

A task notifies the handler:

```
- name: Deploy SSH configuration
  ansible.builtin.template:
    src: sshd_config.j2
    dest: /etc/ssh/sshd_config
  notify:
    - Restart SSH
```



Templates with Jinja2

Configuration differs per server (hostname, IP, FQDN) — templates fill in the blanks.

templates/motd.j2

```
Welcome to {{ inventory_hostname }}
OS: {{ ansible_distribution }}
Kernel: {{ ansible_kernel }}
IP: {{ ansible_default_ipv4.address }}
Managed by Ansible.
```

Deploy it:

```
- name: Configure MOTD
  ansible.builtin.template:
    src: motd.j2
    dest: /etc/motd
    owner: root
    mode: '0644'
```

Variables inside {{ }} are replaced with real values at run time — every server gets a customized result from one file.

9.55 – 9.57

Conditionals, Loops & Tags

Conditionals

```
when:
  "'dns' in group_names"
```

Loops

```
loop:
  - curl
  - wget
  - vim
```

Tags

```
tags:
  - docker
```

Run or skip tasks selectively by tag:

```
ansible-playbook playbooks/site.yml --tags docker
ansible-playbook playbooks/site.yml --skip-tags docker
```

- Conditionals target tasks at specific hosts or groups
- Loops remove duplicated tasks for repeated items
- Tags let large infrastructures run only what's needed — saving time during development

Role Best Practices & Summary

- One service per role
- Keep tasks small and focused
- Prefer Ansible modules over shell commands
- Use variables instead of hardcoded values
- Restart services only through handlers
- Use tags to organize large playbooks
- Use FQCN module names
- Write idempotent tasks

Where We Stand

Each infrastructure component is now its own reusable role:

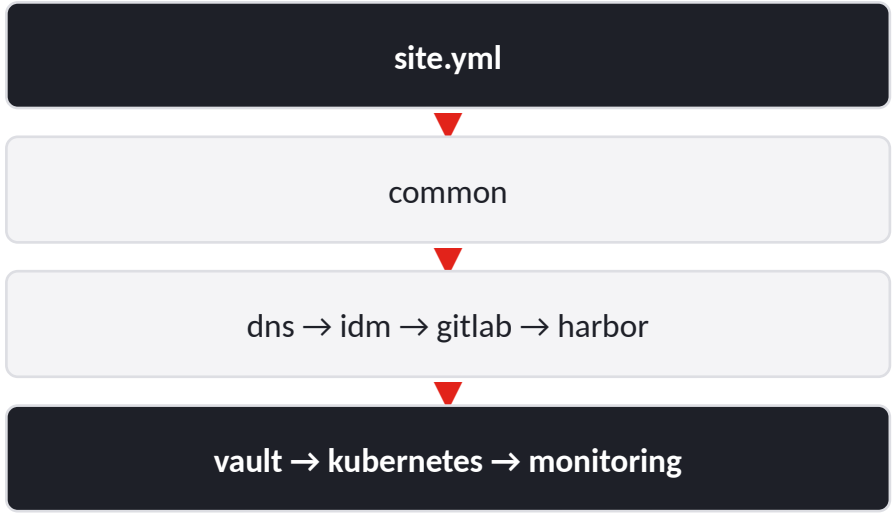
```
roles/
├── common/
├── dns/   ├── idm/
├── edge/  ├── vault/
├── gitlab/ ├── harbor/
├── runner/
├── kubernetes/
├── monitoring/
└── siem/
```

PART V

Case Study: Building the Common Role

Applying everything so far to a real, shared baseline used by every server in the lab.

The Common Role's Responsibilities



The common role runs first — every server begins from the same known configuration.

Task	Description
Update Packages	Latest security updates
Install Utilities	Common admin tools
Configure Timezone	Consistent timestamps
Configure Chrony	Time synchronization
Configure SSH	Secure remote admin
Configure MOTD	Login banner
Auto Security Updates	unattended-upgrades

Directory Structure & Task Includes

```
roles/common/
├── defaults/main.yml
├── handlers/main.yml
├── tasks/
│   ├── main.yml
│   ├── packages.yml
│   ├── users.yml
│   ├── ssh.yml
│   ├── chrony.yml
│   └── firewall.yml
└── templates/
    ├── sshd_config.j2
    └── chrony.conf.j2
```

tasks/main.yml imports each focused file:

```
---
- ansible.builtin.import_tasks: packages.yml
- ansible.builtin.import_tasks: users.yml
- ansible.builtin.import_tasks: ssh.yml
- ansible.builtin.import_tasks: chrony.yml
- ansible.builtin.import_tasks: firewall.yml
- ansible.builtin.import_tasks: motd.yml
```

Each task file now has a single responsibility — easy to read, easy to maintain.

9.64

Default Variables — defaults/main.yml

```
---
timezone: UTC
chrony_ntp_servers:
  - pool.ntp.org
packages_common:
  - curl
  - wget
  - vim
  - git
  - chrony
  - ca-certificates
  - unattended-upgrades
ssh_permit_root_login: "no"
ssh_password_authentication: "no"
```

Why defaults/ matters

- Lowest variable precedence — always overridable
- Keeps the role itself untouched
- Future chapters can override any value
- One place to see every configurable setting

9.65 – 9.68

Packages, Timezone & Chrony

tasks/packages.yml

```
- ansible.builtin.apt:
    update_cache: true
    cache_valid_time: 3600
- ansible.builtin.apt:
    name: "{{ packages_common }}"
    state: present
```

tasks/timezone.yml

```
- community.general.timezone:
    name: "{{ timezone }}"
```

tasks/chrony.yml

```
- ansible.builtin.apt:
    name: chrony
- ansible.builtin.template:
    src: chrony.conf.j2
    dest: /etc/chrony/chrony.conf
  notify: Restart Chrony
```

Time sync matters for: LDAP, Kerberos, TLS, Vault, Kubernetes, Logging.

```
pool {{ chrony_ntp_servers[0] }} iburst
driftfile /var/lib/chrony/chrony.drift
makestep 1.0 3
```

9.69 – 9.71

Administrative Users & Security Baseline

tasks/users.yml

```
- ansible.builtin.group:
    name: devops
- ansible.builtin.user:
    name: ansible
    groups: [sudo, devops]
    create_home: true
```

Authorized key from the controller:

```
- ansible.posix.authorized_key:
    user: ansible
    key: "{{ lookup('file', ...) }}"
```

Setting	Value
Root Login	Disabled
Password Authentication	Disabled
SSH Keys	Required
Automatic Updates	Enabled
CA Certificates	Installed
Chrony	Enabled
MOTD	Customized

9.72 – 9.75

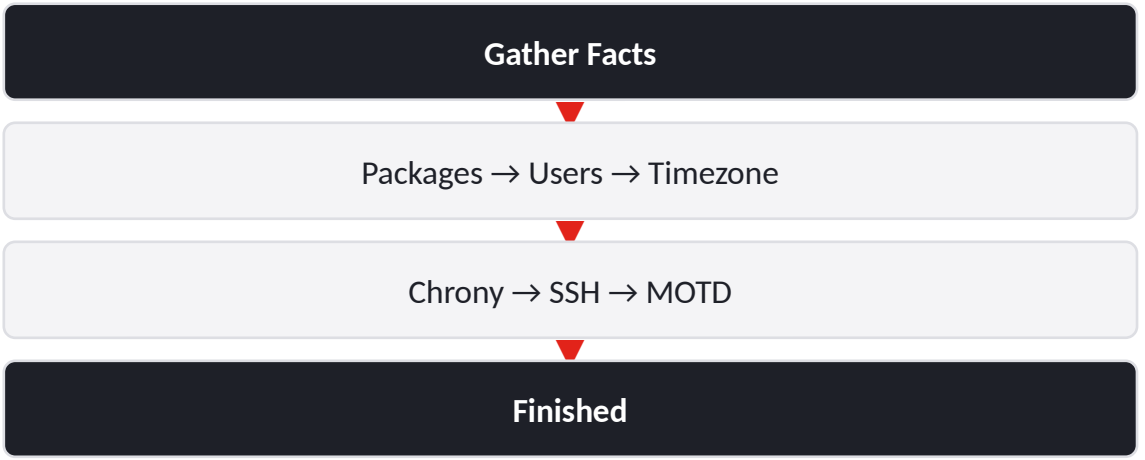
SSH Hardening & Verifying Idempotency

templates/sshd_config.j2

```
PermitRootLogin {{ ssh_permit_root_login }}
PasswordAuthentication {{ ssh_password_authentication
}}
PubkeyAuthentication yes
X11Forwarding no
MaxAuthTries 3
```

Deployed via a template task that notifies “Restart SSH” only on change.

Run it, then run it again:




```
ansible-playbook playbooks/site.yml

changed=0
failed=0
```

changed=0 confirms the role is idempotent.

Best Practices When Designing Roles

- Keep each role focused on one responsibility
- Prefer Ansible modules over shell commands
- Store configurable values in defaults/
- Use templates instead of copying static files
- Trigger service restarts only through handlers
- Test every role independently before integrating
- Ensure every task is idempotent
- Document role variables and dependencies

The common role now provides:

- A standardized Linux baseline
- Secure SSH configuration
- Time synchronization
- Reusable variables
- Modular task organization
- Idempotent execution

PART VI

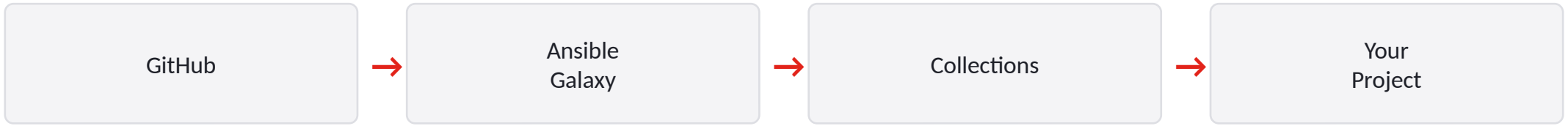
Galaxy, Collections & Quality Tooling

Reusing community automation, and proving the automation works before it ships.

9.77 – 9.79

Ansible Galaxy & Collections

Galaxy is the package manager for Ansible — Roles, Collections, Plugins, Modules, Docs.



Role	Collection
Project component	Package
Executes tasks	Contains modules
Used by playbooks	Used by roles & playbooks
Usually project-specific	Usually shared

Instead of shelling out to Docker:

```
shell:
  docker ps
```

use the collection module — idempotent, supported, tested:

```
community.docker.docker_container
```

Installing Collections via requirements.yml

Collection	Purpose
ansible.posix	Linux administration
community.general	General modules
community.docker	Docker
kubernetes.core	Kubernetes
community.crypto	TLS Certificates
community.hashi_vault	HashiCorp Vault

requirements.yml

```
---
collections:
  - name: ansible.posix
  - name: community.general
  - name: community.docker
  - name: kubernetes.core
  - name: community.crypto
  - name: community.hashi_vault
```

```
ansible-galaxy collection install -r
requirements.yml
```

Every team member receives identical dependencies.

9.83 – 9.85

FQCN & Built-In Documentation

Instead of

```
apt:
service:
copy:
```

use the Fully Qualified Collection Name

```
ansible.builtin.apt:
ansible.builtin.service:
ansible.builtin.copy:
```

- Clear module origin
- No ambiguity between collections

Every module documents itself locally:

```
ansible-doc ansible.builtin.apt
ansible-doc community.docker.docker_container
ansible-doc -l   # list every installed module
```

Ansible includes thousands of modules — you rarely need to search the internet.

Linting Playbooks

Working playbooks aren't enough — professional automation follows coding standards.

```
sudo apt install ansible-lint
ansible-lint playbooks/site.yml

Passed · 0 failures · 0 warnings
```

Corrected — faster, idempotent, easier to understand:

```
ansible.builtin.apt:
  name: nginx
  state: present
```

Consider this task:

```
shell:
apt install nginx

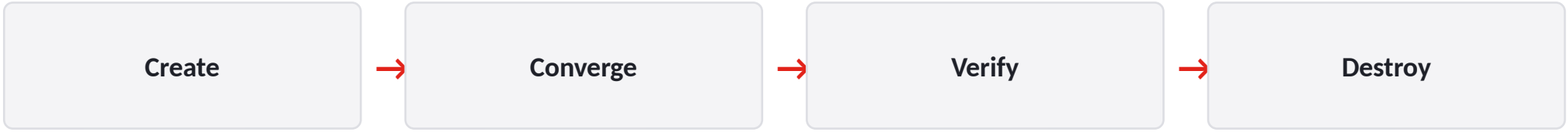
# Ansible Lint reports:
# Use apt module instead of shell.
```

Linting enforces best practices across a team, automatically.

9.88 – 9.90

Testing Roles with Molecule

Molecule creates a temporary environment, executes the role, verifies it, then destroys it.



Install & initialize:

```
python3 -m pip install molecule
molecule --version
cd roles/common
molecule init scenario default
```

Run each stage, or all at once:

```
molecule create
molecule converge
molecule verify
molecule destroy

molecule test   # full workflow
```

Version Control & Chapter Summary

Commit the automation project to Git — small, frequent commits are easier to review and restore.

```
git init
git add .
git commit -m "Initial Ansible automation
framework"
```

In this chapter, we learned to:

- Install & configure Ansible
- Build a production-ready project structure
- Organize hosts into inventories & groups
- Create reusable Roles
- Use variables, facts, templates & handlers
- Apply idempotent automation
- Manage Collections & Galaxy
- Validate with ansible-lint & Molecule
- Manage the project with Git

Chapter 09 — Complete

The Automation Foundation Is Now In Place

Inventory · Roles · Variables · Templates · Handlers · Collections · Testing